

Technology Stack and Architecture

Recommended stack, architecture and deployment

Project	Property Insurance Placement ERP (working title)
Client	Fiducial (Anila Mary S)
Vendor	Spirezen Enterprises (Hiral Patel)
Document version	v0.2 (Draft)
Date	05 October 2026
Status	Draft for client review

Version 0.2 is based on the IIB Code Master workbook, the Sookshma and Laghu add-on charts, and the four client workbooks received on 5 Oct 2026 (Data Sheet, RFQ, QCR, Placement Slip). Items still waiting on client answers are marked **[TBC]**. See document 07 for the analysis of the client formats.

1. Guiding principles

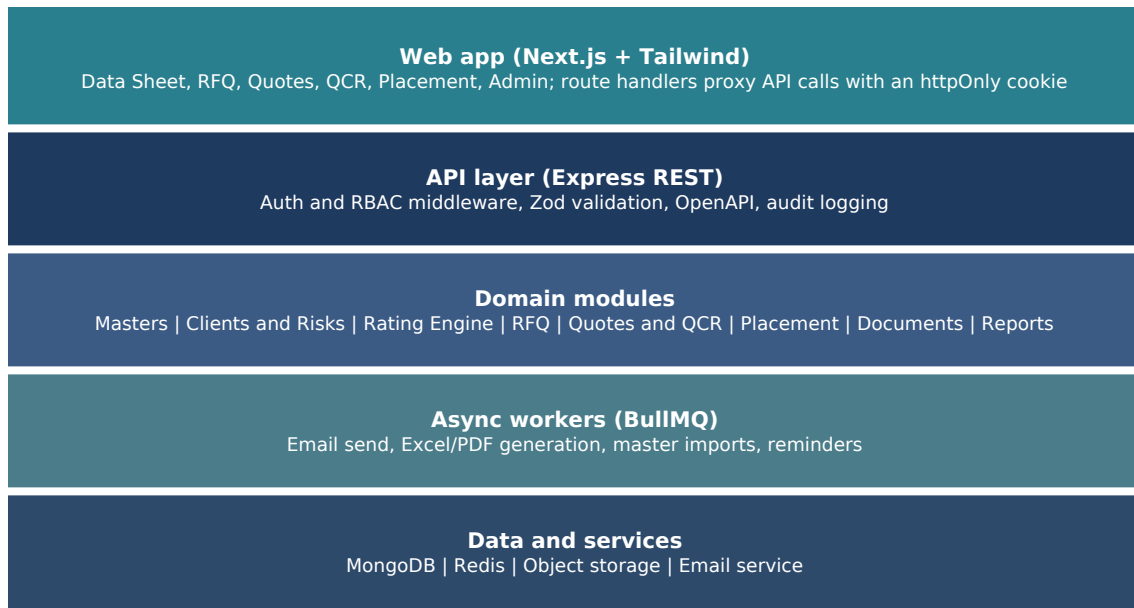
- Correctness first: rating logic is isolated in one tested module with decimal arithmetic.
- Master data is versioned and effective-dated so past quotes stay reproducible.
- Documents (Excel and PDF) are generated from templates so layouts can change without code changes where possible.
- A modular monolith in release 1: simple to deploy and operate, with clear module boundaries so parts can be split out later.
- Hosted in India; infrastructure defined as code.

2. Recommended stack

Layer	Choice	Reason
Frontend	Next.js (App Router), React, TypeScript, Tailwind CSS, shadcn/ui components, TanStack Query, React Hook Form + Zod	Rich data-entry forms and tables; server-side route handlers act as a secure proxy; utility-first styling
Backend	Node.js 20, Express (TypeScript), Zod validation, REST + OpenAPI (zod-to-openapi, Swagger UI), pino logging	Simple, widely known; feature-based modules; shared Zod types with frontend
Database	MongoDB 7 (replica set, Atlas on AWS Mumbai or self-managed)	Document model fits nested risks, sections and quotes; flexible quote terms; multi-document transactions; Decimal128 for money
ODM / migrations	Mongoose with Zod validation, migrate-mongo for versioned data migrations	Typed schemas, validation and repeatable changes
Background jobs	Redis + BullMQ	Email sending, document generation, imports
Excel generation	ExcelJS (template fill)	Fill the client's own workbooks as templates, keeping merged cells and formulas
PDF generation	HTML templates rendered with Playwright/Chromium	Accurate layouts and easy template edits
File storage	S3-compatible object storage	Quote documents, generated files, import files
Email	Amazon SES or client SMTP	Reliable delivery with logs
Auth	JWT sessions with refresh tokens, argon2 password hashing, RBAC guards; OIDC/SSO optional	Secure and extensible
Testing	Vitest, Supertest, mongodb-memory-server (replica set), Playwright, k6 for load	Unit, integration, end-to-end and performance coverage
CI/CD	GitHub Actions, Docker images	Automated build, test and deploy
Hosting	AWS ap-south-1 (Mumbai): ECS Fargate or EC2 (Next.js and Express as separate containers), MongoDB Atlas (ap-south-1) or EC2 replica set, ElastiCache, S3, CloudFront	Data residency in India, managed backups
Observability	Sentry, CloudWatch / Grafana, structured JSON logs	Error tracking and monitoring

Alternative: Python (Django + DRF) on the backend is equally suitable and has strong Excel/PDF libraries. Final choice should follow the team's existing strengths; the architecture below does not depend on it.

3. Logical architecture



4. Key design decisions

Topic	Decision
Rating engine	Pure, side-effect-free module. Input: risk, locations, selected add-ons, master version. Output: premium breakdown. Uses decimal arithmetic and has a golden-file test suite built from client examples.
Master versioning	Each master document has valid_from / valid_to; a master_version document groups an import. Each rating stores the master_version id used.
Add-ons as data	Calculation type (percent of claim, percent of SI, specified SI, escalation, flat on policy SI) plus parameters held in tables; new add-ons need no code if they fit a type.
Documents	Templates stored per document type and version. Excel via template fill; PDF via HTML. Generated files stored and linked to the record.
Audit	Append-only audit collection written in the same multi-document transaction as the change; the application database role has insert and read rights only.
Money and rates	Rates and amounts stored as Decimal128 (rates per mille, amounts to 2 decimals); no floats; calculations use a decimal library in the application.
Search	Unique index on pincode per master version for exact lookup; Atlas Search (or text index) on occupancy description and district.
Multi-tenancy	Single tenant in R1; tables include organisation_id to allow expansion.

MongoDB data modelling approach

Topic	Approach
Embed or reference	Embed data that is read and written together: a risk document embeds its locations, coverage sections, line items, annexure items and selected add-ons; a quote document embeds its quote lines and conditions. Reference shared data: clients, insurers, masters, clauses.
Large masters	occupancies (299) and pincodes (about 20,600) are separate collections with a master_version field; a risk stores a snapshot of the occupancy and pincode values it used, so later master changes never alter past quotes.
Immutable records	rating_runs, audit_logs and generated documents are insert-only collections.
Transactions	Multi-document transactions (replica set required) for status changes, placement locking and audit writes.
Validation	Zod and Mongoose schemas in the application, plus MongoDB JSON Schema validators on critical collections.

Topic	Approach
Reporting	Aggregation pipelines for dashboard and MIS; heavier analytics can later be exported to a warehouse.
Backups	Atlas continuous backup with point-in-time restore, or mongodump plus oplog backup if self-managed.

Trade-off: MongoDB gives flexible, nested documents that match the Data Sheet and quote matrix well, but offers less built-in referential integrity and more work for cross-collection reports than a relational database. Integrity rules therefore live in the service layer and are covered by tests.

5. Environments and deployment

Environment	Purpose	Data
Local	Developer machines using Docker Compose	Seed data from masters
Dev	Integration testing	Synthetic
Staging	Client demos and UAT	Masters plus anonymised sample risks
Production	Live use	Live; daily backups, point-in-time recovery

6. Security and compliance

- TLS 1.2+ in transit, encryption at rest (database, object storage, backups).
- Least-privilege IAM, secrets in a secrets manager, no credentials in code.
- Dependency and container scanning in CI; penetration test before go-live.
- Role-based access plus record ownership checks; all exports are audited.
- Personal data minimised and handled per India's DPDP Act; retention rules agreed with client.

7. Estimated running cost drivers

Compute (a Next.js container, an Express API container and one worker), a MongoDB replica set (Atlas or self-managed), Redis, object storage, email, and monitoring. A cost estimate will be prepared once hosting preference (client AWS account or vendor-managed) is confirmed.